Manual:Scripting

Applies to RouterOS: any



Scripting language manual

This manual provides introduction to RouterOS built-in powerful scripting language.

Scripting host provides a way to automate some router maintenance tasks by means of executing user-defined scripts bounded to some event occurrence.

Scripts can be stored in Script repository or can be written directly to console. The events used to trigger script execution include, but are not limited to the System Scheduler, the Traffic Monitoring Tool, and the Netwatch Tool generated events.

Line structure

RouterOS script is divided into number of command lines. Command lines are executed one by one until the end of script or until runtime error occur.

Command line

RouterOS console uses following command syntax:

`[prefix] [path] command [uparam] [param=[value]] .. [param=[value]]`

- [prefix] - ":" or "/" character which indicates if command is ICE or path. May or may not be required.
- [path] - relative path to the desired menu level. May or may not be required.
- command - one of the commands available at the specified menu level.
- [uparam] - unnamed parameter, must be specified if command requires it.
- [params] - sequence of named parameters followed by respective values

The end of command line is represented by the token *";"* or *NEWLINE*. Sometimes *";"* or *NEWLINE* is not required to end the command line.

Single command inside `(), [] or {}` does not require any end of command character. End of command is determined by content of whole script

```
:if ( true ) do={ :put "lala" }
```

Each command line inside another command line starts and ends with square brackets "[ ]" (command concatenation).

```
:put [/ip route get [find gateway=1.1.1.1]];
```

Notice that code above contains three command lines:

- :put
- /ip route get
- find gateway=1.1.1.1

Command line can be constructed from more than one physical line by following line joining rules.

Physical Line

A physical line is a sequence of characters terminated by an end-of-line (EOL) sequence. Any of the standard platform line termination sequences can be used:

- **unix** – ASCII LF;
- **windows** – ASCII CR LF;
- **mac** – ASCII CR;

Standard C conventions for new line characters can be used (the \n character).

Comments

A comment starts with a hash character (#) and ends at the end of the physical line. Whitespace or any other symbols are not allowed before hash symbol. Comments are ignored by syntax. If (#) character appear inside string it is not considered a comment.

```
# this is a comment
 # bad comment
:global a; # bad comment

:global myStr "lala # this is not a comment"
```

Line joining

Two or more physical lines may be joined into logical lines using backslash character (\). A line ending in a backslash cannot carry a comment. A backslash does not continue a comment. A backslash does not continue a token except for string literals. A backslash is illegal elsewhere on a line outside a string literal.

```
:if ($a = true \
      and $b=false) do={ :put “$a $b”; }

:if ($a = true \      # bad comment
      and $b=false) do={ :put “$a $b”; }

# comment \
    continued – invalid  (syntax error)
```

Whitespace between tokens

Whitespace can be used to separate tokens. Whitespace is necessary between two tokens only if their concatenation could be interpreted as a different token. Example:

```
{
   :local a true; :local b false;
# whitespace is not required
   :put (a&&b);
# whitespace is required
   :put (a and b);
}
```

Whitespace are not allowed

- between '<parameter>='
- between 'from=' 'to=' 'step=' 'in=' 'do=' 'else='

Example:

```
#incorrect:
:for i from = 1 to = 2 do = { :put $i }
#correct syntax:
:for i from=1 to=2 do={ :put $i }
:for i from= 1 to= 2 do={ :put $i }

#incorrect
/ip route add gateway = 3.3.3.3
#correct
/ip route add gateway=3.3.3.3
```

Scopes

Variables can be used only in certain regions of the script. These regions are called scopes. Scope determines visibility of the variable. There are two types of scopes - `global` and `local`. A variable declared within a block is accessible only within that block and blocks enclosed by it, and only after the point of declaration.

Global scope

Global scope or root scope is default scope of the script. It is created automatically and can not be turned off.

Local scope

User can define its own groups to block access to certain variables, these scopes are called local scopes. Each local scope is enclosed in curly braces ("{ }").

```
{
   :local a 3;
   {
      :local b 4;
      :put ($a+$b);
   }
#line below will generate error
   :put ($a+$b);
}
```

In code above variable `b` has local scope and will not be accessible after closed curly brace.



Note: Each line written in terminal is treated as local scope

So for example, defined local variable will not be visible in next command line and will generate syntax error

```
[admin@MikroTik] > :local myVar a;
[admin@MikroTik] > :put $myVar
syntax error (line 1 column 7)
```



Warning: Do not define global variables inside local scopes.

Note that even variable can be defined as global, it will be available only from its scope unless it is not already defined.

```
{
   :local a 3;
   {
       :global b 4;
   }
   :put ($a+$b);
}
```

Code above will generate an error.

Keywords

The following words are keywords and cannot be used as variable and function names:

```
and       or        not       in
```

Delimiters

The following tokens serve as delimiters in the grammar:

```
()  []  {}  :  ;  $  /
```

Data types

RouterOS scripting language has following data types:

Type	Description
num (number)	- 64bit signed integer, possible hexadecimal input;
bool (boolean)	- values can bee `true` or `false`;
str (string)	- character sequence;
ip	- IP address;
ip6-prefix	- IPv6 prefix
id (internal ID)	- hexadecimal value prefixed by '*' sign. Each menu item has assigned unique number - internal ID;
time	- date and time value;
array	- sequence of values organized in an array;
nil	- default variable type if no value is assigned;

Constant Escape Sequences

Following escape sequences can be used to define certain special character within string:

\"	Insert double quote
****	Insert backslash
\n	Insert newline
\r	Insert carriage return
\t	Insert horizontal tab
\$	Output $ character. Otherwise $ is used to link variable.
\?	Output ? character. Otherwise ? is used to print "help" in console.
_	- space
\a	- BEL (0x07)
\b	- backspace (0x08)
\f	- form feed (0xFF)
\v	Insert vertical tab
\xx	Print character from hex value. Hex number should use capital letters.

```
:put "\48\45\4C\4C\4F\r\nThis\r\nis\r\na\r\ntest";
```

which will show on display

```
HELLO
This
is
a
test
```

Operators

Arithmetic Operators

Usual arithmetic operators are supported in RouterOS scripting language

Opearator	Description	Example
"+"	binary addition	`:put (3+4);`
"-"	binary subtraction	`:put (1-6);`
"*"	binary multiplication	`:put (4*5);`
"/"	binary division	`:put (10 / 2); :put ((10)/2)`
"-"	unary negation	`{ :local a 1; :put (-a); }`



Note: for division to work you have to use braces or spaces around dividend so it is not mistaken as IP address

Relational Operators

Opearator	Description	Example
"<"	less	`:put (3<4);`
">"	greater	`:put (3>4);`
"="	equal	`:put (2=2);`
"<="	less or equal	
">="	greater or equal	
"!="	not equal	

Logical Operators

Opearator	Description	Example
"!", "not"	logical NOT	`:put (!true);`
"&&", "and"	logical AND	`:put (true&&true)`
"||", "or"	logical OR	`:put (true||false);`
"in"		`:put (1.1.1.1/32 in 1.0.0.0/8);`

Bitwise Operators

Bitwise operators are working on number and ip address data types.

Opearator	Description	Example
"~"	bit inversion	`:put (~0.0.0.0)`
"|"	bitwise OR. Performs logical OR operation on each pair of corresponding bits. In each pair the result is "1" if one of bits or both bits are "1", otherwise the result is "0".	
"^"	bitwise XOR. The same as OR, but the result in each position is "1" if two bits are not equal, and "0" if bits are equal.	
"&"	bitwise AND. In each pair the result is "1" if first and second bit is "1". Otherwise the result is "0".	
"<<"	left shift by given amount of bits	
">>"	right shift by given amount of bits	

Concatenation Operators

Opearator	Description	Example
“.”	concatenates two strings	`:put (“concatenate” . “ “ . “string”);`
“,”	concatenates two arrays or adds element to array	`:put ({1;2;3} , 5 );`

It is possible to add variable values to strings without concatenation operator:

```
:global myVar "world";

:put ("Hello " . $myVar);
# next line does the same as above
:put "Hello $myVar";
```

By using $[] and $() in string it is possible to add expressions inside strings:

```
:local a 5;
:local b 6;
:put " 5x6 = $($a * $b)";

:put " We have $[ :len [/ip route find] ] routes";
```

Other Operators

Opearator	Description	Example
“`[]`”	command substitution. Can contain only single command line	`:put [ :len "my test string"; ];`
“`()`”	sub expression or grouping operator	`:put ( "value is " . (4+5));`
“`$`”	substitution operator	`:global a 5; :put $a;`
“`~`”	binary operator that matches value against POSIX extended regular expression	Print all routes which gateway ends with 202 `/ip route print where gateway~"^[0-9\\.]*202"`

“`->`”	Get an array element by key	`[admin@x86] >:global aaa {a=1;b=2}`
		`[admin@x86] > :put ($aaa->"a")`
		`1`
		`[admin@x86] > :put ($aaa->"b")`
		`2`

Variables

Scripting language has two types of variables:

- `global` - accessible from all scripts created by current user, defined by global keyword;
- `local` - accessible only within the current scope, defined by local keyword.



Note: Starting from v6.2 there can be undefined variables. When variable is undefined parser will try to look for variables set, for example, by DHCP `lease-script` or Hotspot `on-login`

Every variable, except for built in RouterOS variables, must be declared before usage by local or global keywords. Undefined variables will be marked as undefined and will result in compilation error. Example:

```
# following code will result in compilation error, because myVar is used without declaration
:set myVar "my value";
:put $myVar
```

Correct code:

```
:local myVar;
:set myVar "my value";
:put $myVar;
```

Exception is when using variables set, for example, by DHCP `lease-script`

```
/system script
add name=myLeaseScript policy=\
    ftp,reboot,read,write,policy,test,winbox,password,sniff,sensitive,api \
    source=":log info \$leaseActIP\r\
    \n:log info \$leaseActMAC\r\
    \n:log info \$leaseServerName\r\
    \n:log info \$leaseBound"

/ip dhcp-server set  myServer lease-script=myLeaseScript
```

Valid characters in variable names are letters and digits. If variable name contains any other character, then variable name should be put in double quotes. Example:

```
#valid variable name
:local myVar;
#invalid variable name
:local my-var;
#valid because double quoted
:global "my-var";
```

If variable is initially defined without value then variable data type is set to *nil*, otherwise data type is determined automatically by scripting engine. Sometimes conversion from one data type to another is required. It can be achieved using data conversion commands. Example:

```
#convert string to array
:local myStr "1,2,3,4,5";
:put [:typeof $myStr];
:local myArr [:toarray $myStr];
:put [:typeof $myArr]
```

Variable names are case sensitive.

```
:local myVar "hello"
# following line will generate error, because variable myVAr is not defined
:put $myVAr
# correct code
:put $myVar
```

Set command without value will un-define the variable (remove from environment, new in v6.2)

```
#remove variable from environment
:global myVar "myValue"
:set myVar;
```

Commands

Every global command should start with *":"* token, otherwise it will be treated as variable.

Command	Syntax	Description	Example
/		go to root menu	
..		go back by one menu level	
?		list all available menu commands and brief descriptions	
global	:global <var> [<value>]	define global variable	:global myVar "something"; :put $myVar;
local	:local <var> [<value>]	define local variable	{ :local myLocalVar "I am local"; :put $myVar; }
beep	:beep <freq> <length>	beep built in speaker	
delay	:delay <time>	do nothing for a given period of time	
put	:put <expression>	put supplied argument to console	
len	:len <expression>	return string length or array element count	:put [:len "length=8"];
typeof	:typeof <var>	return data type of variable	:put [:typeof 4];
pick	:pick <var> <start>[<end>]	return range of elements or substring. If end position is not specified, will return only one element from an array.	:put [:pick "abcde" 1 3]
log	:log <topic> <message>	write message to system log. Available topics are "debug, error, info and warning"	:log info "Hello from script";
time	:time <expression>	return interval of time needed to execute command	:put [:time {:for i from=1 to=10 do={ :delay 100ms }}];
set	:set <var> [<value>]	assign value to declared variable.	:global a; :set a true;
find	:find <arg> <arg> <start>	return position of substring or array element	:put [:find "abc" "a" -1];
environment	:environment print <start>	print initialized variable information	:global myVar true; :environment print;
terminal		terminal related commands	
error	:error <output>	Generate console error and stop executing the script	
parse	:parse <expression>	parse string and return parsed console commands. Can be used as function.	:global myFunc [:parse ":put hello!"]; $myFunc;
resolve	:resolve <arg>	return IP address of given DNS name	:put [:resolve "www.mikrotik.com"];
toarray	:toarray <var>	convert variable to array	
tobool	:tobool <var>	convert variable to boolean	
toid	:toid <var>	convert variable to internal ID	
toip	:toip <var>	convert variable to IP address	
toip6	:toip6 <var>	convert variable to IPv6 address	
tonum	:tonum <var>	convert variable to integer	

tostr	:tostr <var>	convert variable to string
totime	:totime <var>	convert variable to time

Menu specific commands

Following commands available from most sub-menus:

Command	Syntax	Description
add	add <param>=<value>..<param>=<value>	add new item
remove	remove <id>	remove selected item
enable	enable <id>	enable selected item
disable	disable <id>	disable selected item
set	set <id> <param>=<value>..<param>=<value>	change selected items parameter, more than one parameter can be specified at the time. Parameter can be unset by specifying '!' before parameter. Example: /ip firewall filter add chain=blah action=accept protocol=tcp port=123 nth=4,2 print set 0 !port chain=blah2 !nth protocol=udp
get	get <id> <param>=<value>	get selected items parameter value
print	print <param><param>=[<value>]	print menu items. Output depends on print parameters specified. Most common print parameters are described here
export	export [file=<value>]	export configuration from current menu and its sub-menus (if present). If file parameter is specified output will be written to file with extension '.rsc', otherwise output will be printed to console. Exported commands can be imported by import command
edit	edit <id> <param>	edit selected items property in built-in text editor
find	find <expression>	Returns list of internal numbers for items that are matched by given expression. For example: :put [/interface find name~"ether"]

import

Import command is available from root menu and is used to import configuration from files created by export command or written manually by hand.

print parameters

Several parameters are available for print command:

Parameter	Description	Example
append		
as-value	print output as array of parameters and its values	`:put [/ip address print as-value]`
brief	print brief description	
detail	print detailed description, output is not as readable as brief output, but may be useful to view all parameters	
count-only	print only count of menu items	
file	print output to file	
follow	print all current entries and track new entries until ctrl-c is pressed, very useful when viewing log entries	`/log print follow`
follow-only	print and track only new entries until ctrl-c is pressed, very useful when viewing log entries	`/log print follow-only`
from	print parameters only from specified item	`/user print from=admin`
interval	continuously print output in selected time interval, useful to track down changes where `follow` is not acceptable	`/interface print interval=2`
terse	show details in compact and machine friendly format	
value-list	show values one per line (good for parsing purposes)	
without-paging	If output do not fit in console screen then do not stop, print all information in one piece	
where	expressions followed by where parameter can be used to filter out matched entries	`/ip route print where interface="ether1"`

More than one parameter can be specified at a time, for example, `/ip route print count-only interval=1 where interface="ether1"`

Loops and conditional statements

Command	Syntax	Description
do..while	`:do { <commands> } while=( <conditions> ); :while ( <conditions> ) do={ <commands> };`	execute commands until given condition is met.
for	`:for <var> from=<int> to=<int> step=<int> do={ <commands> }`	execute commands over a given number of iterations
foreach	`:foreach <var> in=<array> do={ <commands> };`	execute commands for each elements in list

Command	Syntax	Description
if	`:if(<condition>) do={<commands>} else={<commands>} <expression>`	If a given condition is `true` then execute commands in the `do` block, otherwise execute commands in the `else` block if specified.

Example:

```
{
   :local myBool true;
   :if ($myBool = false) do={ :put "value is false" } else={ :put "value is true" }
}
```

Functions

Scripting language does not allow to create functions directly, however you could use :parse command as a workaround.

Starting from v6.2 new syntax is added to easier define such functions and even pass parameters. It is also possible to return function value with **:return** command.

See examples below:

```
#define function and run it
:global myFunc do={:put "hello from function"}
$myFunc

output:
hello from function
```

```
#pass arguments to the function
:global myFunc do={:put "arg a=$a"; :put "arg '1'=$1"}
$myFunc a="this is arg a value"  "this is arg1 value"

output:
arg a=this is arg a value
arg '1'=this is arg1 value
```

Notice that there are two ways how to pass arguments:

- pass arg with specific name ("a" in our example)
- pass value without arg name, in such case arg "1", "2" .. "n" are used.

Return example

```
:global myFunc do={ :return ($a + $b)}
:put [$myFunc a=6 b=2]

output:
8
```

You can even clone existing script from script environment and use it as function.

```
#add script
 /system script add name=myScript source=":put \"Hello $myVar !\""
```

```
:global myFunc [:parse [/system script get myScript source]]
$myFunc myVar=world

output:
Hello world !
```



Warning: If function contains defined global variable which name matches the name of passed parameter, then globally defined variable is ignored, for compatibility with scripts written for older versions. This feature can change in future versions. **Avoid using parameters with same name as global variables.**

For example:

```
:global my2 "123"

:global myFunc do={ :global my2; :put $my2; :set my2 "lala"; :put $my2 }
$myFunc my2=1234
:put "global value $my2"
```

Output will be:

```
1234
lala
global value 123
```

Nested function example



Note: to call another function its name needs to be declared (the same as for variables)

```
:global funcA do={ :return 5 }
:global funcB do={
  :global funcA;
  :return ([$funcA] + 4)
}
:put [$funcB]


Output:
9
```

Catch run-time errors

Starting from v6.2 scripting has ability to catch run-time errors.

For example, [code]:reslove[/code] command if failed will throw an error and break the script.

```
[admin@MikroTik] > { :put [:resolve www.example.com]; :put "lala";}
failure: dns name does not exist
```

Now we want to catch this error and proceed with our script:

```
:do {
      :put [:resolve www.example.com];
} on-error={ :put "resolver failed"};
:put "lala"

output:

resolver failed
lala
```

Operations with Arrays



Warning: Key name in array contains any character other than lowercase character, it should be put in quotes

For example:

```
[admin@ce0] > {:local a { "aX"=1 ; ay=2 }; :put ($a->"aX")}

2
```

Loop through keys and values

foreach command can be used to loop through keys and elements:

```
[admin@ce0] > :foreach k,v in={2; "aX"=1 ; y=2; 5} do={:put ("$k=$v")}

0=2
1=5
aX=1
y=2
```



Note: If array element has key then these elements are sorted in alphabetical order, elements without keys are moved before elements with keys and their order is not changed (see example above).

Change the value of single array element

```
[admin@MikroTik] > :global a {x=1; y=2}
[admin@MikroTik] > :set ($a->"x") 5
[admin@MikroTik] > :environment print
a={x=5; y=2}
```

Script repository

Sub-menu level: `/system script`

Contains all user created scripts. Scripts can be executed in several different ways:

- **on event** - scripts are executed automatically on some facility events (scheduler, netwatch, VRRP)
- **by another script** - running script within script is allowed
- **manually** - from console executing run command or in winbox

Property	Description
name (*string*; Default: *"Script[num]"*)	name of the script
policy (*string*; Default:)	list of applicable policies: • **api** - api permissions • **ftp** - can log on remotely via ftp and send and retrieve files from the router • **local** - can log on locally via console • **password** - change passwords • **policy** - manage user policies, add and remove user • **read** - can retrieve the configuration • **reboot** - can reboot the router • **sensitive** - see passwords and other sensitive information • **sniff** - can run sniffer, torch etc • **ssh** - can log on remotely via secure shell • **telnet** - can log on remotely via telnet • **test** - can run ping, traceroute, bandwidth test • **web** - can log on remotely via http • **winbox** - winbox permissions • **write** - can retrieve and change the configuration Read more detailed policy descriptions here
source (*string*;)	Script source code

Read only status properties:

Property	Description
last-started (*date*)	Date and time when the script was last invoked.
owner (*string*)	User who created the script
run-count (*integer*)	Counter that counts how many times script has been executed

Menu specific commands

Command	Description
run (*run [id|name]*)	Execute specified script by ID or name

Environment

Sub-menu level:

- `/system script environment`
- `/environment`

Contains all user defined variables and their assigned values.

```
[admin@MikroTik] > :global example;
[admin@MikroTik] > :set example 123
[admin@MikroTik] > /environment print
"example"=123
```

Read only status properties:

Property	Description
name (*string*)	Variable name
user (*string*)	User who defined variable
value ()	Value assigned to variable

Job

Sub-menu level: `/system script job`

Contains list of all currently running scripts.
Read only status properties:

Property	Description
owner (*string*)	User who is running script
policy (*array*)	List of all policies applied to script
started (*date*)	Local date and time when script was started

Article Sources and Contributors

Manual:Scripting *Source*: http://wiki.mikrotik.com/index.php?oldid=26069 *Contributors*: Burek, Janisk, Marisb, Normis, Proofreader

Image Sources, Licenses and Contributors

Image:Version.png *Source*: http://wiki.mikrotik.com/index.php?title=File:Version.png *License*: unknown *Contributors*: Normis

Image:Icon-note.png *Source*: http://wiki.mikrotik.com/index.php?title=File:Icon-note.png *License*: unknown *Contributors*: Marisb, Route

Image:Icon-warn.png *Source*: http://wiki.mikrotik.com/index.php?title=File:Icon-warn.png *License*: unknown *Contributors*: Marisb, Route